

Week 5 & 6

AI-Guided Fuzzing Framework - Progress Report

Week 5 - My Planning and Design Work

In Week 5, my plan was to design the AI-guided input generation approach before implementing the fuzzer. I prepared the workflow so model output could be generated, cleaned, executed, logged, measured, and compared with the random fuzzer.

Prompt -> Language Model -> Generated Inputs -> Validation -> Execution
-> Logging -> Coverage Measurement -> Summary Generation

What I Created in Week 5

I created the design materials needed for AI input generation. These included the generation plan, prompt categories, output validation strategy, methodology updates, and Week 6 implementation plan.

Deliverable	What I used it for
AI input generation design	I defined how the model would generate calculator fuzzing inputs.
Prompt library	I planned prompts for valid, invalid, division-by-zero, parentheses-heavy, large-number, and mixed inputs.
Output validation design	I planned how to remove explanations, blank lines, numbering, duplicates, and invalid formats.
Methodology updates	I updated the research method to include AI-generated inputs and validation.
Week 6 implementation plan	I prepared the next step: connecting the model to the fuzzer.

Model Choice

I selected qwen3:1.7b through Ollama because it runs locally and supports reproducible experiments. I kept larger models as future work, but I started with a smaller local model so the framework could be tested consistently.

Week 5 Outcome

By the end of Week 5, I had a clear AI-guided fuzzing plan. The main lesson was that validation is necessary because raw model output cannot always be used directly as fuzzing input.

Week 6 - Implementation, Testing, and Fair Comparison

AI-guided fuzzer integration and final 100 vs 100 comparison

What I Implemented

In Week 6, I followed the Week 5 plan and implemented the real AI-guided fuzzing pipeline. I replaced the old hardcoded AI fuzzer prototype with a pipeline that connects to Ollama, generates inputs, validates them, executes them, logs the results, summarizes the run, and compares the output with the random fuzzer.

File	Purpose
src/utlis/ai_generator.py	I used this to request calculator inputs from qwen3:1.7b through Ollama.
src/utlis/ai_validator.py	I used this to clean model output and keep calculator-like expressions.
src/fuzzers/ai_fuzzer.py	I replaced the hardcoded prototype with the generator + validator pipeline.
run_ai_fuzzer.py	I used this runner to execute and log AI-generated inputs.
summarize_ai_results.py	I used this to summarize AI fuzzing results.
experiments/compare_results.py	I used this to compare random fuzzing and AI-guided fuzzing.

Testing and Fixes

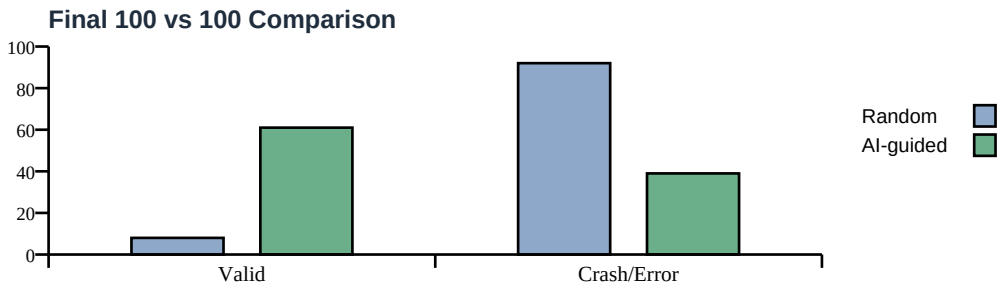
My first working AI experiment generated 20 inputs: 18 valid and 2 crash/error inputs. That showed the AI fuzzer could generate meaningful expressions, but it was not fair because the random fuzzer used 100 inputs.

I then tried 100 inputs in one model request, but qwen3:1.7b timed out after 120 seconds. I changed the experiment to smaller batches of 20 inputs per round. That worked, but one run produced 121 inputs, so I fixed the runner to stop at exactly 100 executed inputs.

```
TARGET_INPUT_COUNT = 100
MAX_GENERATION_ATTEMPTS = 10
# I stop logging as soon as 100 AI inputs have been executed.
```

Final Fair Comparison Result

Metric	Random Fuzzer	AI-Guided Fuzzer
Total Inputs	100	100
Valid Inputs	8	61
Crash/Error Inputs	92	39
Average Execution Time	0.00002734 sec	0.00003274 sec



My Main Finding

After the final fix, both fuzzers used exactly 100 inputs. The AI-guided fuzzer generated 61 valid inputs, while the random fuzzer generated only 8. This shows that AI-guided fuzzing was better at producing structured calculator expressions.

The random fuzzer produced more crash/error inputs and more error types, including `SyntaxError`, `ValueError`, `KeyError`, `IndentationError`, and `ZeroDivisionError`. My conclusion is that random fuzzing is useful for broad malformed-input exploration, while AI-guided fuzzing is useful for targeted and structured edge-case generation.

Week 7 - Pilot Experiments and Paper Drafting

50 vs 50 pilot run, workflow debugging, and paper draft deliverables

What I Did in Week 7

In Week 7, I started pilot experiments using the completed random and AI-guided fuzzing pipeline. I used a smaller 50 vs 50 setup so I could check the workflow before scaling the experiments further.

- I ran a 50-input random fuzzing pilot and a 50-input AI-guided pilot.
- I saved Week 7 results separately so the Week 6 100 vs 100 comparison would not be overwritten.
- I created a pilot summary script to automatically compare both result logs.
- I drafted the System Architecture and Implementation Details paper sections.

Pilot Setup

Item	Value
Random inputs	50
AI-guided inputs	50
Target	calculator evaluator
Model	qwen3:1.7b via Ollama

Workflow Debugging

My first Week 7 summary script was wrong because it counted all inputs as errors. I fixed the script so it correctly reads the JSONL format and reports valid inputs, crash/error inputs, unique inputs, duplicate inputs, execution time, and error types.

```
PILOT_INPUT_COUNT = 50
RANDOM_OUTPUT_FILE = "week7_random_pilot_results.jsonl"
AI_OUTPUT_FILE = "week7_ai_pilot_results.jsonl"
```

Final Week 7 Pilot Result

Metric	Random	AI-guided
Total Inputs	50	50
Valid Inputs	3	42
Crash/Error Inputs	47	8
Unique Inputs	47	50
Duplicate Inputs	3	0
Avg Time	0.00003119s	0.00002818s

Week 7 50 vs 50 Pilot Comparison



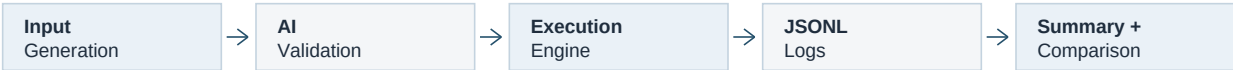
Error Types

- Random: SyntaxError 34, ValueError 7, ZeroDivisionError 4, KeyError 1, IndentationError 1.
- AI-guided: ZeroDivisionError 8.

Paper Drafting Deliverables

- I created a system architecture figure showing generation, validation, execution, logging, summary, and comparison.
- I drafted the System Architecture section.
- I drafted the Implementation Details section.

Architecture Flow I Documented



My Main Finding

The Week 7 pilot confirmed the same pattern as Week 6. The AI-guided fuzzer produced far more valid calculator inputs: 42 out of 50, compared with only 3 out of 50 for the random fuzzer. The random fuzzer still produced more diverse malformed-input errors, while the AI-guided fuzzer produced targeted division-by-zero cases. This supports my conclusion that AI-guided fuzzing is strong for structured inputs, while random fuzzing is useful for broad malformed-input exploration.